

Stl Sort

排序相关 STL

本页面将简要介绍 C 和 C++ 标准库中实现的排序算法。

除已说明的函数外，本页所列函数默认定义于头文件 `<algorithm>` 中。

qsort

参见：[qsort](#)，[std::qsort](#)

该函数为 C 标准库实现的 [快速排序](#)，定义在 `<stdlib.h>` 中。在 C++ 标准库里，该函数定义在 `<cstdlib>` 中。

qsort 与 bsearch 的比较函数

qsort 函数有四个参数：数组名、元素个数、元素大小、比较规则。其中，比较规则通过指定比较函数来实现，指定不同的比较函数可以实现不同的排序规则。

比较函数的参数限定为两个 `const void` 类型的指针。返回值规定为正数、负数和 0。

比较函数的一种示例写法为：

```
1 int compare(const void *p1, const void *p2) // int 类型数组的比较函数
2 {
3     int *a = (int *)p1;
4     int *b = (int *)p2;
5     if (*a > *b)
6         return 1; // 返回正数表示 a 大于 b
7     else if (*a < *b)
8         return -1; // 返回负数表示 a 小于 b
9     else
10         return 0; // 返回 0 表示 a 与 b 等价
11 }
```

注意：返回值用两个元素相减代替正负数是一种典型的错误写法，因为这样可能会导致溢出错误。

以下是排序结构体的一个示例：

```

1 struct eg // 示例结构体
2 {
3     int e;
4     int g;
5 };
6
7 int compare(const void *p1,
8             const void *p2) // struct eg 类型数组的比较函数：按成员 e 排序
9 {
10     struct eg *a = (struct eg *)p1;
11     struct eg *b = (struct eg *)p2;
12     if (a->e > b->e)
13         return 1; // 返回正数表示 a 大于 b
14     else if (a->e < b->e)
15         return -1; // 返回负数表示 a 小于 b
16     else
17         return 0; // 返回 0 表示 a 与 b 等价
18 }

```

这里也可以看出，等价不代表相等，只代表在此比较规则下两元素等价。

std::sort

参见：[std::sort](#)

用法：

```

1 // a[0] .. a[n - 1] 为需要排序的数列
2 // 对 a 原地排序，将其按从小到大的顺序排列
3 std::sort(a, a + n);
4
5 // cmp 为自定义的比较函数
6 std::sort(a, a + n, cmp);

```

注意：sort 的比较函数的返回值是 true 和 false，用 true 和 false 表示两个元素的大小（先后）关系，这与 qsort 的三值比较函数的语义完全不同。具体内容详见上方给出的 sort 的文档。

如果要将 sort 简单改写为 qsort，维持排序顺序整体上不变（不考虑等价的元素），需要将返回 true 改为 - 1，返回 false 改为 1。

std::sort 函数是更常用的 C++ 库比较函数。该函数的最后一个参数为二元比较函数，未指定 cmp 函数时，默认按从小到大的顺序排序。

旧版 C++ 标准中仅要求它的平均时间复杂度达到 $O(n^2)$ 。C++11 标准以及后续标准要求它的最坏时间复杂度达到 $O(n \log n)$ 。

C++ 标准并未严格要求此函数的实现算法，具体实现取决于编译器。[libstdc++](#) 和 [libc++](#) 中的实现都使用了 [内省排序](#)。

std::nth_element

参见：[std::nth_element](#)

用法：

```

1 std::nth_element(first, nth, last);
2 std::nth_element(first, nth, last, cmp);

```

它重排 $[first, last)$ 中的元素，使得 nth 所指向的元素被更改为 $[first, last)$ 排好序后该位置会出现的元素。这个

新的 `nth` 元素前的所有元素小于或等于新的 `nth` 元素后的所有元素。

实现算法是未完成的内省排序。

对于以上两种用法，C++ 标准要求它的平均时间复杂度为 $O(n \log n)$ ，其中 n 为 `std::distance(first, last)`。

它常用于构建 [K-D Tree](#)。

`std::stable_sort`

参见：[std::stable_sort](#)

用法：

```
1 std::stable_sort(first, last);
2 std::stable_sort(first, last, cmp);
```

稳定排序，保证相等元素排序后的相对位置与原序列相同。

时间复杂度为 $O(n \log n)$ ，当额外内存可用时，复杂度为 $O(n)$ 。

`std::partial_sort`

参见：[std::partial_sort](#)

用法：

```
1 // mid = first + k
2 std::partial_sort(first, mid, last);
3 std::partial_sort(first, mid, last, cmp);
```

将序列中前 k 元素按 `cmp` 给定的顺序进行原地排序，后面的元素不保证顺序。未指定 `cmp` 函数时，默认按从小到大的顺序排序。

复杂度：约 $k \log k$ 次应用 `cmp`。

原理：

`std::partial_sort` 的思想是：对原始容器内区间为 `[first, mid)` 的元素执行 `make_heap()` 操作，构造一个大根堆，然后将 `[mid, last)` 中的每个元素和 `first` 进行比较，保证 `first` 内的元素为堆内的最大值。如果小于该最大值，则互换元素位置，并对 `[first, mid)` 内的元素进行调整，使其保持最大堆序。比较完之后，再对 `[first, mid)` 内的元素做一次堆排序 `sort_heap()` 操作，使其按增序排列。注意，堆序和增序是不同的。

自定义比较

参见：[运算符重载](#)

内置类型（如 `int`）和用户定义的结构体允许定制调用 STL 排序函数时使用的比较函数。可以在调用该函数时，在最后一个参数中传入一个实现二元比较的函数。

对于用户定义的结构体，对其使用 STL 排序函数前必须定义至少一种关系运算符，或是在使用函数时提供二元比较函数。通常推荐定义 `operator<`。¹

示例：

```

1 int a[1009], n = 10;
2 // ...
3 std::sort(a + 1, a + 1 + n); // 从小到大排序
4 std::sort(a + 1, a + 1 + n, greater<int>()); // 从大到小排序

1 struct data {
2     int a, b;
3
4     bool operator<(const data rhs) const {
5         return (a == rhs.a) ? (b < rhs.b) : (a < rhs.a);
6     }
7 } da[1009];
8
9 bool cmp(const data u1, const data u2) {
10     return (u1.a == u2.a) ? (u1.b > u2.b) : (u1.a > u2.a);
11 }
12
13 // ...
14 std::sort(da + 1, da + 1 + 10); // 使用结构体中定义的 < 运算符, 从小到大排序
15 std::sort(da + 1, da + 1 + 10, cmp); // 使用 cmp 函数进行比较, 从大到小排序

```

严格弱序

另请参阅: [C++ 中的应用 - 序理论](#)

进行排序的运算符必须满足 [严格弱序](#), 否则会出现不可预料的情况 (如运行时错误、无法正确排序)。

常见的错误做法:

- 使用 `<=` 来定义排序中的小于运算符。
- 在调用排序运算符时, 读取外部数值可能会改变的数组 (常见于最短路算法)。
- 将多个数的最大最小值进行比较的结果作为排序运算符 (如皇后游戏/加工生产调度 中的经典错误)。

外部链接

- [浅谈邻项交换排序的应用以及需要注意的问题](#)

参考资料与注释

1. 因为大部分标准算法默认使用 `operator<` 进行比较。↩

本页面最近更新: 2024/9/20 22:28:51, [更新历史](#)

发现错误? 想一起完善? [在 GitHub 上编辑此页!](#)

本页面贡献者: [billchenchina](#), [ouuan](#), [Alisahhh](#), [caijianhong](#), [Enter-tainer](#), [Great-designer](#), [lr1d](#), [k rn1pnc](#), [ksyx](#), [NachtgeistW](#), [partychicken](#), [Tiphereth-A](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供, 附加条款亦可能应用